COURSE-BASED PROJECT DOCUMENTATION | DOC-02

Objectives, Scope & Requirements

Problem Formulation, Motivations, & Functional Boundaries

1. Problem Statement

Java 21 expanded the grammar with intricate constructs such as record patterns, deconstruction, and contextual guards. Adapting these constructs into a single-token lookahead LALR(1) grammar introduces complex shift/reduce and reduce/reduce ambiguities. There is a need for a formally verified, deterministic LALR(1) Java 21 syntax analyzer coupled with a modern graphical interface capable of pinpoint error-line highlighting.

2. Project Motivation

- ? Apply formal Language Theory (DFAs, CFGs, LALR(1) tables) to modern Java 21 LTS syntax.
- ? Solve classic compiler conflicts: dangling-else, cast expressions, ragged arrays, and switch guards.
- ? Build a zero-overhead native C engine communicating with Python via in-memory ctypes FFI.
- ? Deliver a polished, dark-themed VS Code-style developer desktop environment.

3. Core Project Objectives

- ? Objective 1: Construct a complete GNU Bison LALR(1) grammar for Java 21 (LTS).
- ? Objective 2: Build a fast GNU Flex lexical scanner with line and column tracking.
- ? Objective 3: Provide exact, drift-free error line reporting across all formatting scenarios.
- ? Objective 4: Export a C shared library (javacheck.dll) and standalone CLI (javacheck.exe).
- ? Objective 5: Develop a Python Tkinter GUI with Command Palette and error highlighting.
- ? Objective 6: Validate 100% test passing rate across 59 parser tests and 14 GUI tests.

4. Supported vs. Non-Supported Functionality

Functional Domain	Supported in Project	Non-Supported / Excluded
Lexical Tokens	Keywords, Literals, Comments, Text ...	Preprocessor macros (Java has none)
Grammar Syntax	Java 21 LTS Finalized Standard	Experimental preview syntax (JEP 430)
Error Detection	First syntax error line & token near	Multi-error speculative recovery ca...
Semantics	Grammar rule conformance	Type checking, symbol scope resolution
Execution	Syntax validation status report	Bytecode generation, JVM execution